

Client-Server Model & Request Lifecycle

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

English:

Every system you will ever design in an interview is a variation of one idea: a client asks for something, a server does the work and answers back. This is the client-server model, and it is the single most important mental model in all of system design.

Hindi:

जो भी system आप कभी interview में design करेंगे, वह असल में एक ही idea का रूप है: एक client कुछ मांगता है, एक server वह काम करके जवाब देता है। यही client-server model है, और पूरे system design में यही सबसे ज़रूरी mental model है।

Important Words:

- **Mental model** – मानसिक मॉडल / सोचने का ढांचा

English:

A client is anything that initiates a request: a mobile app, a browser, another backend service. A server is anything that listens for requests and returns a response. Critically, "server" doesn't mean one physical machine — in real systems it's usually a fleet of machines behind a load balancer, but from the client's point of view, it still behaves like a single logical server.

Hindi:

Client वह कुछ भी है जो request शुरू करता है: एक mobile app, एक browser, या कोई दूसरी backend service। Server वह कुछ भी है जो requests सुनता है और response देता है। सबसे ज़रूरी बात, "server" का मतलब एक physical machine नहीं है — असली systems में यह आमतौर पर load balancer के पीछे कई machines का एक समूह होता है, लेकिन client के नज़रिए से यह फिर भी एक logical server जैसा ही behave करता है।

Important Words:

- **Initiate** – शुरू करना
- **Fleet (of machines)** – मशीनों का समूह
- **Logical** – लॉजिकल / तार्किक रूप से

Backend engineer analogy

English:

This is literally the Laravel/Express request lifecycle you already know: a browser sends an HTTP request, it hits your router, your controller runs, it may query MySQL or hit Redis, and a response goes back. System design just zooms out: instead of one server, imagine 500 servers, and instead of one client, imagine 50 million phones.

Hindi:

यह बिल्कुल वही Laravel/Express request lifecycle है जो आप पहले से जानते हैं: एक browser एक HTTP request भेजता है, वह आपके router तक पहुंचती है, आपका controller चलता है, वह शायद MySQL query करे या Redis को hit करे, और फिर एक response वापस जाता है। System design बस इसे जूम-आउट करता है: एक server की जगह 500 servers सोचें, और एक client की जगह 5 करोड़ फोन सोचें।

Important Words:

- **Zoom out (figurative)** – बड़े स्तर पर देखना

The full request lifecycle, step by step

English:

When you type a URL and press Enter, roughly this happens: DNS resolution, TCP connection, TLS handshake (if HTTPS), HTTP request sent, server processing, HTTP response sent back, and the client renders or consumes the response. In an HLD interview, you rarely need to describe every step in detail, but you need to know they exist.

Hindi:

जब आप एक URL टाइप करके Enter दबाते हैं, तो लगभग यह होता है: DNS resolution, TCP connection, TLS handshake (अगर HTTPS हो), HTTP request भेजी जाती है, server processing होती है, HTTP response वापस भेजी जाती है, और client उस response को render या consume करता है। HLD interview में आपको हर step को detail में बताने की ज़रूरत शायद ही पड़े, लेकिन आपको यह पता होना चाहिए कि ये steps मौजूद हैं।

Important Words:

- **Consume (data)** – उपयोग करना

English:

One subtlety worth internalizing early: the client-server model itself says nothing about how many machines are behind "server." Whether the request lands on a monolith or a mesh of 40 microservices, the client's experience is identical. This is why interviewers can ask you to "design X" without specifying microservices vs. monolith upfront.

Hindi:

एक बारीक बात जैसी शुरुआत में ही समझ लेना अच्छा है: client-server model खुद यह कुछ नहीं बताता कि "server" के पीछे कितनी machines हैं। चाहे request एक monolith पर जाए या 40 microservices के जाल पर, client का experience एक जैसा ही रहता है। यही वजह है कि interviewer बिना पहले से microservices या monolith बताए "design X" पूछ सकते हैं।

Important Words:

- **Monolith** – मोनोलिथ / एक बड़ा संगठित एप्लिकेशन
- **Mesh (figurative)** – जाल / आपस में जुड़ा हुआ ढांचा

- **Upfront** – पहले से

English:

One more nuance: this lifecycle describes a single request. Real user sessions involve dozens of these round trips in sequence, and each one independently goes through DNS lookup (usually cached after the first), a connection (often reused via HTTP keep-alive), and the request/response cycle. Connections can be reused, not re-established from scratch every time.

Hindi:

एक और बारीक बात: यह lifecycle सिर्फ एक request के बारे में बताता है। असली user sessions में ऐसे दर्जनों round trips क्रम में होते हैं, और हर एक अपने आप DNS lookup (जो पहली बार के बाद cache हो जाता है), एक connection (जो अक्सर HTTP keep-alive से reuse होता है), और request/response cycle से गुज़रता है। Connections को हर बार नए सरि से नहीं, बल्कि reuse किया जा सकता है।

Important Words:

- **Round trip** – राउंड ट्रिप / आना-जाना
- **Reused** – फरि से उपयोग किया गया

2. Real-World Example / Case Study

English:

When you open the PhonePe app to pay a merchant, the app (client) sends a request to PhonePe's backend (server) to initiate a transaction. That backend is not one machine — it's dozens of services behind load balancers, but the app only ever "talks" to what looks like a single API endpoint.

Hindi:

जब आप किसी merchant को pay करने के लिए PhonePe app खोलते हैं, तो app (client) PhonePe के backend (server) को एक request भेजता है ताकि transaction शुरू हो। वह backend एक machine नहीं है — यह load balancers के पीछे दर्जनों services होती हैं, लेकिन app हमेशा सिर्फ एक जैसे दिखने वाले single API endpoint से "बात" करता है।

Important Words:

- **Endpoint** – एंडपॉइंट

English:

Careem's ride-booking flow is another clean example: the rider's app (client) sends a booking request; Careem's backend matches a driver, and the response streams back updates over time. Note this already hints at a limitation of plain request-response: booking status changes after the initial response.

Hindi:

Careem का ride-booking flow एक और साफ उदाहरण है: rider का app (client) एक booking request भेजता है; Careem का backend एक driver match करता है, और उसके बाद समय के साथ updates आते रहते हैं। यह पहले से ही plain request-response की एक कमी दिखाता है: booking का status शुरुआती response के बाद भी बदलता रहता है।

Important Words:

- **Hints at (figurative)** – इशारा करना

English:

A third example: Meesho's seller dashboard. When a seller updates a product's stock count, the browser sends a straightforward synchronous request, and the server returns success or failure — a textbook, low-drama use of the client-server model with no real-time requirement at all.

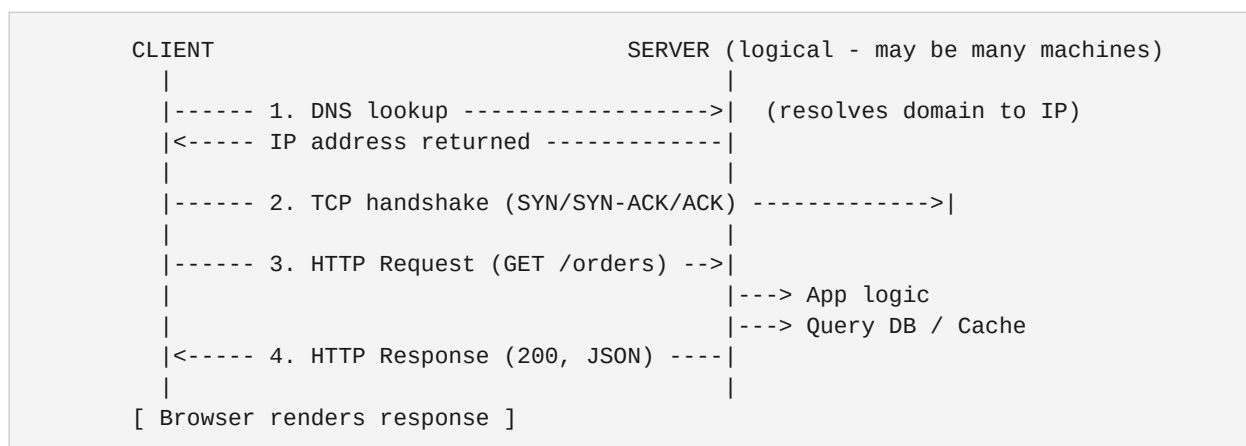
Hindi:

एक तीसरा उदाहरण: Meesho का seller dashboard। जब कोई seller किसी product की stock count update करता है, तो browser एक सीधा synchronous request भेजता है, और server success या failure लौटाता है — यह client-server model का एक बिल्कुल simple इस्तेमाल है जिसमें कोई real-time ज़रूरत ही नहीं है।

Important Words:

- **Straightforward** – सीधा / सरल

3. Diagram (English labels kept as-is)



4. Trade-offs Table (Reference Table – English)

Pattern	When to use	Limitation
Plain request-response (HTTP)	Simple CRUD, most REST APIs	Server can't push updates to client on its own
Polling (client asks repeatedly)	Near-real-time needs, simple to build	Wasteful; wastes requests when nothing changed
Long polling	Better than polling, moderate real-time needs	Still holds connections open; harder to scale
WebSockets (full-duplex)	True real-time: chat, live tracking	More complex infra; covered fully in Phase 3

5. Common Interview Questions

Q1: "Walk me through what happens when a user opens your app's home screen."

English:

Model approach: Narrate the lifecycle top to bottom -- DNS, connection, request, load balancer, app server, cache/DB lookup, response. Don't over-explain DNS/TCP in depth unless asked; spend more time on what happens once the request reaches your system.

Hindi:

Model approach: पूरे lifecycle को ऊपर से नीचे तक बताएं -- DNS, connection, request, load balancer, app server, cache/DB lookup, response। जब तक ना पूछा जाए, DNS/TCP को बहुत ज्यादा detail में मत समझाएं; ज्यादा समय इस पर दें कि request आपके system तक पहुंचने के बाद क्या होता है।

Q2: "Is HTTP good enough for a chat application? Why or why not?"

English:

Model approach: Explain that plain HTTP is client-initiated only, so the server can't push a new message to a client without the client asking. Mention polling and long-polling as stopgaps, then correctly name WebSockets as the standard solution.

Hindi:

Model approach: बताएं कि plain HTTP सिर्फ client-initiated होता है, इसलिए server बिना client के पूछे उसे नया message push नहीं कर सकता। Polling और long-polling को temporary उपाय के रूप में बताएं, फिर सही तरीके से WebSockets को standard solution के रूप में बताएं।

Important Words:

- **Stopgap** – अस्थायी हल

Q3: "What happens if the TCP connection drops mid-request?"

English:

Model approach: Explain that the client typically detects the failure via a timeout or connection-reset error, and the application layer must decide how to react -- retry the whole request, resume from a checkpoint, or surface an error to the user. Mention idempotency becomes critical here.

Hindi:

Model approach: बताएं कि client आमतौर पर timeout या connection-reset error से failure detect करता है, और application layer को यह decide करना होता है कि क्या करना है -- पूरी request दोबारा भेजे, किसी checkpoint से resume करें, या user को error दिखाएं। यहाँ idempotency बहुत ज़रूरी हो जाती है।

Important Words:

- **Timeout** – टाइमआउट
- **Checkpoint** – चेकपॉइंट
- **Idempotency** – आइडेमपोटेन्सी
- **Surface (verb)** – दिखाना / सामने लाना

6. Practice Exercises

English:

- Write out, in your own words, the 7-step request lifecycle from memory.
- For a food delivery app, identify one feature that fits plain request-response and one that does not, and explain why.
- Sketch the diagram from this chapter but replace 'SERVER' with a load balancer and two app servers.

Hindi:

- 7-step request lifecycle को अपने शब्दों में, याद से लिखें।
- एक food delivery app के लिए, एक ऐसा feature बताएं जो plain request-response में फिट बैठता है और एक जो नहीं बैठता, और वजह भी बताएं।

- इस chapter का diagram बनाएं लेकिन 'SERVER' की जगह एक load balancer और दो app servers रखें।

7. Curated YouTube Videos (Reference – English)

Language	Video & Channel	Why it helps
English	"Web Application Architecture: Full Request-Response Lifecycle"	Walks through the complete lifecycle end to end.
English	"Client-Server Architecture Explained: Request-Response Model"	Clean, visual explanation.
Hindi	"Client Server model in Hindi"	Direct Hindi explanation.
Hindi	"Client Server Model (Distributed Systems, Hindi)"	Same model, distributed-systems framing.

8. Key Takeaways

English:

- Client-server is the foundational pattern underneath every system you'll design in this program.
- "Server" is a logical concept -- in production it's usually many machines behind a load balancer.
- The full request lifecycle: DNS -> TCP handshake -> (TLS) -> HTTP request -> server processing -> HTTP response.
- Plain HTTP is client-initiated; the server cannot push data on its own.
- Polling, long-polling, and WebSockets exist specifically to work around that limitation.
- Interviewers use 'what happens when you open the app' to probe how deep your systems knowledge goes.

Hindi:

- Client-server हर उस system का आधार है जो आप इस program में design करेंगे।
- "Server" एक logical concept है -- production में यह आमतौर पर load balancer के पीछे कई machines होती है।
- पूरा request lifecycle: DNS -> TCP handshake -> (TLS) -> HTTP request -> server processing -> HTTP response।
- Plain HTTP सिर्फ client-initiated होता है; server खुद से data push नहीं कर सकता।
- Polling, long-polling, और WebSockets इसी सीमा को पार करने के लिए बने हैं।
- Interviewers 'app खोलने पर क्या होता है' पूछकर यह test करते हैं कि आपकी systems knowledge कतिनी गहरी है।

General Words Glossary

A consolidated list of the important English words used throughout this chapter, with Hindi meanings and simple explanations — useful for revising vocabulary after finishing the chapter.

English Word	Hindi Meaning	Simple Explanation
Checkpoint	चेकपॉइंट	A saved point to resume from later
Consume (data)	उपयोग करना	To use or process the received data
Endpoint	एंडपॉइंट	A specific URL/address where an API can be reached
Fleet (of machines)	मशीनों का समूह	A group of machines working together
Hints at (figurative)	इशारा करना	To suggest something indirectly
Idempotency	आइडेमपोटेन्सी	Safe to repeat an action without causing extra side effects
Initiate	शुरू करना	To start something
Logical	लॉजिकल / तार्किक रूप से	In concept, even if not physically true
Mental model	मानसिक मॉडल / सोचने का ढांचा	A simplified way of thinking about how something works
Mesh (figurative)	जाल / आपस में जुड़ा हुआ ढांचा	A network of many connected small services
Monolith	मोनोलिथ / एक बड़ा संगठित एप्लिकेशन	A single, large application built as one unit
Reused	फरि से उपयोग किया गया	Used again instead of creating new
Round trip	राउंड ट्रिप / आना-जाना	A full request-and-response cycle
Stopgap	अस्थायी हल	A temporary solution until something better exists
Straightforward	सीधा / सरल	Simple and direct, without complications
Surface (verb)	दखाना / सामने लाना	To show something to the user
Timeout	टाइमआउट	Giving up after waiting too long for a response
Upfront	पहले से	In advance, before starting
Zoom out (figurative)	बड़े स्तर पर देखना	To look at the bigger picture